

Attention Mechanisms in NLP: Enhancing Computational Efficiency

Miguel Ara Adánez	-	202102043
Alejandro Martínez de Guinea García	-	202113492
Lara Ocón Madrid	-	202115710
Javier Prieto Domínguez	-	202103783
Javier Rojo Llorens	-	202109117

April 2024

Abstract

This document presents the first report or milestone for the final project of the courses "Deep Learning" and "Natural Language Processing" by the students of the third year of the Degree in Mathematical Engineering and Artificial Intelligence. Attention mechanisms enable models to focus on specific parts of input data, improving performance on NLP tasks. However, the computational complexity for long sequences remains a challenge. This project aims to address this by exploring various attention mechanisms.

1 Introduction

This week we have started implementing the algorithms and comparing them against each other. We thought we were going to find the algorithms in the literature already programmed to have a frame of reference for our own implementations, but we haven't found all of them. We have decided to try and create a library with code that not only improves the traditional approach of the attention mechanism, but is easy to understand and implement.

2 Sparse Attention Mechanisms

When looking at reducing the computational cost of performing all the attention operations, a first approach which seems quite logic is to reduce the number of tokens the attention mechanism focuses on. An innovative approach to do so is the Big Bird method [2], which combines three attention mechanisms: random attention, global attention and local window attention. We will be focusing just on the latter, as we believe it is the best one to combine with Kernelized attention in the near future.

2.1 Local Window Attention[3]

Given a fixed window size w , each token attends to $\frac{1}{2}w$ tokens on each side. The computation complexity of this pattern is therefore $O(n \cdot w)$, which scales linearly with input sequence length n . In order to implement this model, two different approaches have been developed and compared to each other. The first, and most simple approach, consists in iterating through the sequence to grab the tokens within the specified window size and perform the necessary operations. The second one is far more efficient, but slightly more complex and difficult to understand, as it uses torch's fold and unfold functions to eliminate the need for loops.

Although both models are obtaining adequate results in the test dataset we are working with (around 0.9 accuracy), the first one needs 100 seconds per epoch whereas the second one needs around 15 seconds. With these outstanding results we can state that our sparse attention model could possibly be a substitute for the general self attention, which is slightly slower (17 seconds per epoch).

3 Kernelized Attention

This method deals with traditional Transformer’s attention issue with long sequences, as the softmax function within the attention mechanism has quadratic computational complexity. The kernelized attention method stands out by transforming the softmax function into a form that allows for an efficient matrix multiplication between kernelized features of queries and key-value pairs, leading to linear complexity with respect to sequence length.

The algorithm mainly consists in inverting the order of traditional attention dot-then-exponentiate approach, reducing the amount of exponential operations done: instead of doing an exponential to each query-key pair (n^2), it does an exponential to each query and key individually ($2n$). Then, it computes the dot product between each key-value and finally compares that to the query given. [5]

As to the results obtained, it improves computational time from 17 seconds per epoch of traditional self attention to 14 seconds per epoch, without losing accuracy with the test dataset. Therefore, we concluded that this approach is also a possible substitute for the general self attention in order to solve it’s complexity issue.

4 Linformer: An Efficient Attention Framework

Transformers calculate attention with a quadratic complexity of $O(n^2)$, where n is the sequence length. This is costly for long sequences. The Linformer [6] approximates the full attention matrix with a low-rank matrix to reduce this complexity.

The core idea is to project the sequence-length dimensions of the key and value matrices into a lower dimension, k , using projection matrices E and F . These projection matrices are not learned during training but are initialized based on some predefined rules and kept constant.

$$\begin{aligned} \text{head}_i &= \text{Attention}(QW_i^Q, E_iKW_i^K, F_iVW_i^V) \\ &= \underbrace{\text{softmax}\left(\frac{QW_i^Q(E_iKW_i^K)^T}{\sqrt{d_k}}\right)}_{P:n \times k} \cdot F_iVW_i^V \end{aligned}$$

The matrices E and F have dimensions that effectively reduce the keys and values from $n \times d$ to $n \times k$ and $k \times d$, respectively. The probability of the low-rank approximation being close to the full attention matrix is given by:

$$\Pr(\|\exp(A_iE^T)FV - \exp(A_i)V\| \leq \epsilon \|\exp(A_i)V\|) > 1 - \delta$$

The reduced complexity with the low-rank approximation is $O(nk)$, and since k is independent of n , the new complexity becomes linear with respect to n , that is $O(n)$.

This significant reduction in complexity allows the Linformer to be more efficient, particularly with long sequences. In practice, we’ve tested to see how much it improves with long sequences compared to traditional attention mechanisms, and it speeds up the process by up to six times. However, the improvement is not as significant for shorter sequences, and there is a slight decrease in accuracy. We will continue testing to determine how much better the Linformer performs in terms of speed and accuracy and assess whether it is truly worth using over the standard attention mechanism. The balance between computational efficiency and model performance is crucial, and ongoing experimentation will help clarify the contexts in which the Linformer’s trade-offs are beneficial.

5 Locality-sensitivity hashing

The standard attention used in the Transformer is the scaled dot-product attention. Considering Q , K and V as a matrices of the packed queries, keys and values respectively:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where d_k is the dimension of queries, keys and values.

However, regarding memory, as Q , K and V all have the shape $[batch_size, n, d_k]$ (n referring to the sequence length), the issue is the term QK^T , which has the shape $[batch_size, n, n]$. This would take a great amount of memory, quadratically related to the sequence length.

A way of avoiding it is creating buckets in which we introduce our queries according to the position in the hyperspace they occupy [7]. This means that two vectors that are close to each other are more likely to be sorted into the same bucket than two vectors that are dissimilar. This bucket sort will generally leave the vectors with most similarity in the same bucket, allowing to not consider the rest of queries when computing the QK^T operation, saving space in memory.

The function that maps queries onto buckets is, as of now, dependent on random hyperplanes, and therefore non-deterministic, which gives us better and worse accuracy results in equal manner. The benchmark has confirmed this hypothesis, showing up to a 50% decrease in memory usage with respect to traditional self attention.

6 Future planning

One of our main lines of future development is to combine the Local Window Attention mechanism with the improvements obtained with Kernelized Attention. Having already obtained valuable individual models and results, we believe the combination of these two approaches could result in an innovative Attention model. In parallel, we will be focusing on perfecting the Locality-sensitivity hashing technique as well as writing the mathematics behind each of the attention models.

7 Benchmarks

This paper aims to measure the reduction in complexity of the attention mechanisms proposed. We have created a benchmark to study the time and memory reduction (if there is any) of these mechanisms compared to the one proposed in [1].

This benchmark consists of a training and inference tasks on a big dataset (about 2500 words on average) where given two attention mechanism, and using the same small model, time and memory used are measured over 10 epochs. We can also compare the time with a smallest dataset (68 of maximum sequence length) as well as the accuracy achieved by a bigger model implementing the attention mechanism.

References

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention is all you need*. <https://arxiv.org/abs/1706.03762>
- [2] Zaheer, M., Guruganesh, G., Dubey, A., Ainslie, J., Alberti, C., Ontanon, S., Pham, P., Ravula, A., Wang, Q., Yang, L., & Ahmed, A. (2020). *Big Bird: Transformers for Longer Sequences*. <https://arxiv.org/abs/2007.14062>
- [3] Beltagy, I., Peters, M. E., & Cohan, A. (2020). *Longformer: The Long-Document Transformer*. <https://arxiv.org/abs/2004.05150v2>
- [4] Luo, S., Li, S., Cai, T., He, D., Peng, D., Zheng, S., Ke, G., Wang, L., & Liu, T. (2021). *Stable Fast and Accurate: Kernelized Attention with Relative Positional Encoding*. <https://arxiv.org/abs/2106.12566>
- [5] Peng, H., Pappas, N., Yogatama, D., Schwartz, R., Smith, N. A., & Kong, L. (2021). *Random feature attention*. <https://arxiv.org/abs/2103.02143>
- [6] Wang, S., Li, B. Z., Khabsa, M., Fang, H., & Ma, H. (2020). *Linformer: Self-Attention with Linear Complexity*. <https://arxiv.org/abs/2006.04768>
- [7] Kitaev, N., Kaiser, L., & Levskaya, A. (n.d.). *Reformer: the efficient transformer*. <https://openreview.net/forum?id=rkgNKkHtvB>